

TECHNICAL REFERENCE

Technical Documentation

Multi-tenant biometric attendance and retail platform —
firmware, backend, and dashboard

WHO IT'S FOR

Developers and the platform
operator

VERSION

Version 3.1 | September 2026 |
Firmware 1.10.0

IN THIS GUIDE

- 1 Project overview
- 2 System architecture
- 3 Hardware
- 4 Firmware
- 5 Network and security
- 6 Database schema
- 7 Provisioning and operation
- 8 Edge functions
- 9 Dashboard
- 10 OTA firmware updates
- 11 Key file locations

THE DOCUMENTATION SET

Platform admin

Super admin

Admin

Staff

School

Office

Shop

Privacy note

Technical

Contents

1. Project overview	2
1.1 Key capabilities	2
2. System architecture	3
2.1 Components	3
2.2 Data flow for one scan	3
3. Hardware	4
3.1 Bill of materials	4
3.2 Carrier PCB	4
3.3 Enclosure	4
3.4 Wiring	5
4. Firmware	5
4.1 Task model	5
4.2 Offline queue	5
4.3 OLED display	6
4.4 Server-reachability watchdog	6
4.5 SPIFFS layout	7
5. Network and security	7
5.1 TLS certificate validation	7
5.2 Authentication secrets	8
5.3 Tenant isolation	8
6. Database schema	8
6.1 Core tables	9
6.2 Retail and loyalty tables (shops)	10
7. Provisioning and operation	10
7.1 First boot of a new scanner	10
7.2 Normal operation	10
7.3 Enrollment job lifecycle	11
7.4 Revocation and decommissioning	11
8. Edge functions	11
9. Dashboard	12
9.1 Roles and scope	12
9.2 Pages	13
9.3 Installable PWA	13
10. OTA firmware updates	13
11. Key file locations	14

1 Project overview

The ESP32 Fingerprint Attendance System is a multi-tenant biometric attendance and retail platform. ESP32 scanners with an R503 capacitive fingerprint sensor record attendance at the unit level — a classroom, an office section, a shop counter. A Next.js dashboard backed by one cloud Supabase project provides reporting, device provisioning, remote enrollment, institution configuration, and — for shop tenants — a point-of-sale and loyalty module.

The system replaces paper roll-calls with a tamper-resistant fingerprint scan. Records are timestamped in the institution's own timezone; absences are filled in automatically each evening by a scheduled job. Scanners keep working through WiFi outages by queueing scans to on-board flash.

1.1 Key capabilities

- **Biometric identification** — R503 capacitive sensor, up to 127 fingerprint slots per scanner, on-sensor matching in about 0.3 s.
- **Multi-tenant** — one deployment serves any number of school, office, and shop institutions with strict per-tenant isolation.
- **Per-member-type scan modes** — present / absent (one scan marks the day) or time in / time out (paired scans), set separately for students and staff.
- **Punctuality** — optional late-arrival and early-departure thresholds stamp each scan `on_time`, `late`, or `early_leave`.
- **Tracked weekdays** — an explicit Monday-to-Sunday set; scans on untracked days are ignored and generate no absences.
- **Offline resilience** — every scan is written to SPIFFS before any network attempt; a rotate-then-process queue survives reboots and power loss.
- **Automatic absence marking** — `pg_cron` invokes `mark-absent` daily; it respects holidays, tracked weekdays, and periods.
- **On-device OLED** — an optional SSD1306 128 x 64 screen shows a live status card; the firmware runs headless without it.
- **Remote enrollment** — jobs queued on the dashboard are collected, executed, and reported by the scanner; stuck jobs are re-delivered and self-heal.
- **OTA updates** — scanners check GitHub Releases at boot (after a random delay) and install a strictly newer build.
- **Academic calendar** — terms or periods, plus recurring and one-off holidays.
- **Retail and loyalty** (shops) — clients, a products-and-services catalog, point of sale, and a punch-card loyalty module with redemption at the till.
- **Role-based access** — six roles with institution- and device-scoped visibility.
- **Installable PWA** — the dashboard installs to a home screen or desktop.

2 System architecture

Three tiers: the ESP32 firmware, the Supabase backend (PostgreSQL, Edge Functions, pg_cron), and the Next.js dashboard on Vercel. Scanners talk only to the edge functions, over HTTPS. The dashboard talks only to PostgreSQL, server-side, through the service role.

2.1 Components

Tier	Technology	Role
ESP32 firmware	Arduino / FreeRTOS, one sketch	Scan, queue to flash, drive the OLED, provision, and self-update over HTTPS
R503 sensor	UART, capacitive	Capture, template storage, and matching — entirely on the sensor
DS3231 RTC	I2C	Wall-clock time across reboots and outages; re-synced from NTP on boot and reconnect
SSD1306 OLED	I2C, optional	Status display; shares the I2C bus with the RTC
Supabase Edge Functions	Deno / TypeScript	HTTPS API for provisioning, attendance, enrollment, and the daily absence job
Supabase PostgreSQL	PostgreSQL 17, pg_cron, pg_net	Multi-tenant store; schedules and drives absence marking
Next.js dashboard	App Router, shadcn/ui, Tailwind CSS	The admin interface; all Supabase access server-side
Vercel	Hosting	Dashboard deployment

2.2 Data flow for one scan

- 1 A finger is scanned. The R503 matches it to a stored template and returns a slot number (FID) and a confidence score.
- 2 The firmware resolves the FID to a member through `fid_map.csv`, appends the scan to `/queue.txt` in SPIFFS, and signals the network task.
- 3 The network task renames `/queue.txt` to `/queue_inflight.txt` and POSTs each record to `/log-attendance` with the scanner's `x-device-secret` header.
- 4 `/log-attendance` authenticates the scanner, resolves the member within its institution, applies tracked-weekday, scan-mode, and punctuality rules, deduplicates, and inserts an `attendance` row.
- 5 A trigger bumps `institution_activity.last_change_at`. The dashboard polls `/api/changes` every 12 s and refreshes only when that watermark advances.
- 6 Each evening pg_cron calls `mark-absent` through `net.http_post()`, which inserts absences for every active member with no present scan that day, per institution and timezone.

3 Hardware

A scanner is built from off-the-shelf modules on a custom passive carrier PCB, in a 3D-printed enclosure. Unit cost is roughly GHS 1,000.

3.1 Bill of materials

Item	Part / spec	Notes
Microcontroller	ESP32 dev module — dual-core Xtensa LX6, 240 MHz, 4 MB flash, WiFi	Seats in two 15-pin headers on the carrier
Fingerprint sensor	R503 capacitive — UART, 127 slots, ~0.3 s match, ring LED	6-pin JST-SH 1.0 mm cable
RTC module	DS3231 — I2C, TCXO, battery-backed, +/-2 ppm	6-pin 2.54 mm header; shares I2C with the OLED
Display (optional)	SSD1306 0.96" 128 x 64 OLED, I2C address 0x3C	Firmware runs headless if absent
Carrier PCB	Custom 2-layer interconnect board (KiCad)	No active parts
Enclosure	3D-printed base and lid, plus an OLED bezel variant	FDM, PLA or PETG
Power	5 V USB supply into the carrier's JST-GH 1.25 mm input	3V3 comes from the ESP32 module's regulator

3.2 Carrier PCB

A 2-layer KiCad design in `hardware/PCBs/Attendance_System_v1/`, with an earlier revision in `_v0`. It has no active silicon and no regulator of its own: it seats the ESP32 module and provides keyed, polarised connectors so the sensor, RTC, OLED, and power can't be mis-plugged. It distributes 5 V, 3V3, and GND and routes the UART to the R503 and the shared I2C bus.

Ref	Connector	Purpose
J1 / J2	2 x 1x15, 2.54 mm pin socket	ESP32 dev-module headers
J3	1x6, JST-SH 1.0 mm vertical	R503 sensor — VCC, TX, RX, GND, touch
J4	1x6, 2.54 mm pin socket	DS3231 + SSD1306 — shared SDA/SCL, 3V3, GND; DS3231 32K and SQW broken out
J5	1x2, JST-GH 1.25 mm vertical	5 V power input

The DS3231's 32K and SQW outputs are broken out but unused by the firmware.

3.3 Enclosure

Designed in FreeCAD (`hardware/CAD files/Fingerprint Device Enclosure.FCStd`), exported to 3MF, and sliced in Bambu Studio:

- **Base** — holds the carrier PCB and ESP32; the front face carries the R503 aperture and the power-cable entry.

- **Lid** — closes over the base; the overall envelope is about 80 x 74 x 50 mm.
- **OLED outer shell** — a framed window, about 36 x 35 x 7 mm, for the display on OLED builds.

Print in PLA or PETG. The sensor sits flush behind the front aperture and the lid clips on.

3.4 Wiring

Signal	ESP32 pin
R503 TX → ESP32 RX	GPIO 16
R503 RX → ESP32 TX	GPIO 17
DS3231 and OLED SDA	GPIO 21 (shared I2C bus)
DS3231 and OLED SCL	GPIO 22 (shared I2C bus)

The R503 runs on hardware UART `Serial2` at 8N1. The DS3231 (0x68) and SSD1306 (0x3C) share one bus brought up with `wire.begin(21, 22)`.

4 Firmware

One Arduino sketch: `firmware/ClassAttendance_Current_RTC/ClassAttendance_Current_RTC.ino`, `FIRMWARE_VERSION` 1.10.0. Three FreeRTOS tasks are pinned across the two cores.

4.1 Task model

Task	Core	Responsibility
FingerprintTask	1	Scan loop, FID resolution, LED feedback, enrollment execution, and launching the captive portal on a confirmed master-finger scan. Writes each scan to SPIFFS, then signals NetworkTask.
NetworkTask	0	Provisioning, the queue flush to <code>/log-attendance</code> , NTP-to-RTC sync, WiFi reconnect, and the reachability watchdog (4.4).
EnrollmentTask	0	Polls <code>/get-enrollment-job</code> every 10 s, hands jobs to FingerprintTask, reports results until acknowledged, and acts on the decommission signal.

`memQueueSem`, a binary semaphore, signals "a scan was persisted"; there is no in-RAM scan queue. Enrollment jobs pass through `currentEnrollJob` under `enrollMutex`, and `spiffsMutex` serialises every SPIFFS access. Display state crosses cores through a mailbox — tasks never touch the display object directly.

4.2 Offline queue

- Each scan is written to `/queue.txt` synchronously, inside FingerprintTask, before any network attempt.
- On flush, NetworkTask atomically renames `/queue.txt` to `/queue_inflight.txt` under the lock and releases it, so new scans append to a fresh file while the send runs.
- If power fails mid-flush, `recoverInflightQueue()` merges the orphaned in-flight file back into `/queue.txt` on the next boot, before tasks start.

- A 2xx drops the record. 5xx, 429, and network errors keep it for retry. Any other 4xx — the server answered and rejected the payload — drops it and doesn't count against reachability.
- Caps: 1,000 entries, 256 KB, and 7 days; the oldest records go first, in a sweep every 5 minutes.

Each scan carries a `scan_id` of the form `scan-YYYYMMDDhhmmss-<member-sid>`, timestamped from the DS3231. It is unique per institution and also deduplicated on member, date, and scan type.

4.3 OLED display

If `display.begin()` succeeds, the render path draws a size-2 headline on row 0 and size-1 lines on rows 24, 36, and 48 of the 128 x 64 panel:

- **Idle** — the clock, the device's display name, and its link state: `Online`, `Offline`, or `No server` (WiFi associated but HTTPS failing). A degraded banner on row 48 shows the queue depth and any dropped records.
- **Scan verdicts** — `SCANNED` while awaiting the server, then `PRESENT`, `TIME IN`, `TIME OUT`, or `NOT LOGGED` (accepted but not counted); `SAVED` / Offline when queued; `COOLDOWN`, `NO MATCH`, `NOT MAPPED`, `SCAN ERROR`.
- **Enrollment** — `PRESS 1/2`, `REMOVE`, `PRESS 2/2`, `ENROLLED`, and the failures `MISMATCH`, `TIMED OUT`, `BAD SCAN`, `OCCUPIED`, `STORE FAIL`.
- **Master finger** — `CONFIRM?`, `CONFIRMED`, `CANCELLED`.
- **Lifecycle** — `PENDING` with the MAC, `REJECTED`, `REVOKED`, `RESET`, `SETUP`, and `OTA UPDATE`.

The name on a scan card follows the institution's `member_name_display` setting. The panel sleeps after 120 s idle and wakes on the next event. If it is missing or fails to initialise, every draw is a no-op.

4.4 Server-reachability watchdog

On a live WiFi link, `postJSON*()` feeds a health counter. After 5 consecutive failed POSTs the scanner is "server unreachable": it forces a WiFi re-association every 60 s, and after 5 minutes unreachable it calls `ESP.restart()` to clear a wedged network stack. A 4xx counts as reachable. An unrecoverable TLS-arena or low-heap condition also triggers a reboot.

4.5 SPIFFS layout

File	Contents
<code>/device_identity.json</code>	<code>device_id</code> , <code>institution_id</code> , the per-device <code>device_secret</code> , and <code>display_name</code> , written after assignment
<code>/provisioning.json</code>	<code>device_id</code> and <code>provisioning_token</code> during the pending window; deleted once assigned
<code>/fid_map.csv</code>	<code>fid,uniqueId,role,name</code> — the slot-to-member map used to resolve a scan
<code>/queue.txt</code> , <code>/queue_inflight.txt</code>	The durable attendance queue and its in-flight rotation
<code>/scan_log.txt</code>	A rolling, size-capped local scan log
Device config cache	Per-institution device settings such as <code>member_name_display</code> , with their <code>config_rev</code>
WiFi credentials	The SSID and password captured by the captive portal

5 Network and security

5.1 TLS certificate validation

Every HTTPS connection uses `WiFiClientSecure` with `setCACert(ROOT_CA_BUNDLE)` . The bundle in `certs.h` must contain each host's full chain — root **and** intermediates — because mbedTLS on the ESP32 can't fetch missing intermediates (no AIA).

Host	Chain in the bundle
Supabase (<code>*.supabase.co</code>)	leaf → WE1 (Google Trust Services) → GlobalSign ECC Root CA R4
GitHub API (<code>api.github.com</code>)	leaf → Sectigo Public Server Auth CA DV E36 → Sectigo Root E46
GitHub release CDN	leaf → R12 (Let's Encrypt) → ISRG Root X1 (the root alone suffices)

5.2 Authentication secrets

Secret	Stored	Used for
<code>BOOTSTRAP_SECRET</code>	<code>secrets.h</code> (gitignored, compiled in); edge-function secret	<code>x-bootstrap-secret</code> on <code>/register</code> and <code>/assignment-poll</code> — before assignment only
<code>device_secret</code>	SPIFFS <code>/device_identity.json</code> ; <code>devices.device_secret</code>	<code>x-device-secret</code> on every operational call. A transitional path still accepts the legacy shared <code>institutions.device_secret</code> .
<code>provisioning_token</code>	SPIFFS <code>/provisioning.json</code> ; <code>devices.provisioning_token</code>	Binds <code>/assignment-poll</code> to the scanner that registered, preventing secret harvesting
<code>CRON_SECRET</code>	Supabase Vault (<code>cron_secret</code>) and the edge-function secret	<code>x-cron-secret</code> on <code>/mark-absent</code> . Kong strips the Authorization header, so a Bearer token can't be used.
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Vercel environment	The dashboard's server-side admin client

5.3 Tenant isolation

The dashboard uses the service role, so RLS is bypassed and isolation is enforced in application code. `resolveInstitutionScope()` locks every non-platform role to its own `institution_id` and ignores query parameters; every mutating server action calls `ownsRecord(table, id, session)` before touching a row. `platform_admin` is cross-tenant by design. RLS policies exist as dormant defence in depth. On the device side, `/log-attendance` resolves the member only within the authenticated scanner's institution, so a scan can't cross tenants.

6 Database schema

All tables are in `public` . `institution_id` is the tenant key on every scoped table, with `ON DELETE CASCADE` . Migrations live in `backend/supabase/migrations/` and are applied to cloud by hand after review.

6.1 Core tables

Table	Key columns	Notes
<code>institutions</code>	<code>id</code> , <code>name</code> , <code>type</code> , <code>timezone</code> , <code>label_*</code> , <code>student</code> and <code>staff</code> scan modes, <code>tracked_weekdays</code> , <code>time_format</code> , <code>lateness</code> and <code>early-leave</code> settings, <code>member_name_display</code> , <code>currency</code> , <code>sell_products</code> , <code>sell_services</code> , <code>loyalty_enabled</code> , <code>status</code> , <code>device_secret</code> , <code>config_rev</code> , <code>theme</code>	One row per tenant; type is school, office, or shop. <code>config_rev</code> is bumped by a trigger on any change, and scanners poll it to refresh cached config.
<code>members</code>	<code>id</code> , <code>sid</code> , <code>fullname</code> , <code>group_name</code> , <code>unit_name</code> , <code>fin1</code> , <code>fin2</code> , <code>status</code> , <code>member_type</code> , <code>device_id</code> , <code>institution_id</code>	<code>member_type</code> is student or staff; <code>sid</code> is unique per institution; <code>device_id</code> is <code>ON DELETE SET NULL</code> .
<code>devices</code>	<code>id</code> , <code>group_name</code> , <code>unit_name</code> , <code>display_name</code> , <code>mac</code> , <code>provisioning_token</code> , <code>device_secret</code> , <code>revoked</code> , <code>institution_id</code>	One scanner per institution, group, and unit. <code>display_name</code> is generated from group and unit. <code>revoked = true</code> gives an instant 401.
<code>periods</code>	<code>id</code> , <code>term</code> , <code>year</code> , <code>status</code> , <code>start_date</code> , <code>end_date</code> , <code>institution_id</code>	Academic or reporting periods.
<code>attendance</code>	<code>id</code> , <code>member_id</code> , <code>period_id</code> , <code>device_id</code> , <code>date</code> , <code>time</code> , <code>status</code> , <code>scan_type</code> , <code>punctuality</code> , <code>scan_id</code> , <code>institution_id</code>	<code>status</code> is present or absent; <code>scan_type</code> is present, time_in, or time_out; <code>punctuality</code> is on_time, late, early_leave, or null. Date and time are in the institution's timezone.
<code>enrollment_jobs</code>	<code>id</code> , <code>device_id</code> , <code>member_id</code> , <code>finger_slot</code> , <code>command</code> , <code>status</code> , <code>fid</code> , <code>note</code> , <code>allow_overwrite</code> , <code>dispatched_at</code> , <code>attempts</code> , <code>last_error</code> , <code>institution_id</code> , <code>created_at</code>	<code>command</code> is register, delete, clearall, register-master, or delete-master. A status guard makes completed and failed terminal.
<code>holidays</code>	<code>id</code> , <code>label</code> , <code>start_date</code> , <code>end_date</code> , <code>recurring</code> , <code>institution_id</code>	Recurring holidays match by month and day every year, including ranges that wrap the year end.
<code>profiles</code>	<code>id</code> (→ <code>auth.users</code>), <code>role</code> , <code>assigned_unit</code> , <code>assigned_device_id</code> , <code>institution_id</code> , <code>member_id</code>	Dashboard roles. <code>assigned_device_id</code> is required for teacher and staff and optional for admin.
<code>device_resets</code>	<code>device_id</code> , <code>created_at</code>	Deferred-wipe queue. No foreign key, so it survives device deletion; not consumed on read.
<code>institution_activity</code>	<code>institution_id</code> , <code>last_change_at</code>	Polling watermark maintained by triggers; backs <code>/api/changes</code> .

6.2 Retail and loyalty tables (shops)

Table	Purpose
<code>clients</code>	Customers: name and phone, the phone unique per tenant. Separate from members; no fingerprint.
<code>products</code> , <code>services</code>	Sellable catalog items, soft-deleted through <code>active</code> ; products also carry stock.
<code>client_attendance</code>	One visit row per client per day.
<code>transactions</code> , <code>transaction_items</code>	A sale and its lines. Lines snapshot the item name and unit price. Written atomically by the create-sale function, with discount and reward linkage.
<code>rewards</code>	Loyalty rules: reward kind and payload, condition type, threshold, window, repeatable.
<code>rewards_log</code>	Issuance and redemption records. <code>transaction_id</code> links a redeemed reward to its sale; a guard prevents double redemption.

7 Provisioning and operation

7.1 First boot of a new scanner

- 1 It boots with no `/device_identity.json` . On the very first boot it also clears SPIFFS and restarts.
- 2 It connects to saved WiFi, or opens the captive portal `Attendance-Setup` at 192.168.4.1.
- 3 It syncs time over NTP into the DS3231.
- 4 It POSTs its MAC to `/register` with `x-bootstrap-secret` . The function creates an unassigned `devices` row and returns `device_id` , a `provisioning_token` , and `pending` . The scanner saves both and breathes yellow.
- 5 It polls `/assignment-poll` every 5 s with its id and token.
- 6 The platform admin assigns it to an institution on the Devices page.
- 7 `/assignment-poll` returns `assigned` , the `institution_id` , the per-device `device_secret` , and the display name — only if the token matches.
- 8 The scanner writes `/device_identity.json` , deletes `/provisioning.json` , and enters normal operation. The super admin then gives it a group and unit.

7.2 Normal operation

- **Every scan:** queued to SPIFFS, then POSTed to `/log-attendance` with `x-device-secret` .
- **Enrollment:** `/get-enrollment-job` every 10 s; results POSTed to `/update-enrollment-job` and retried until acknowledged.
- **Configuration:** `/get-enrollment-job` also returns `member_name_display` , the display name, and `config_rev` ; the scanner refreshes its cache when `config_rev` advances.
- **OTA:** checked at boot after a 0–120 s random delay (section 10).

- **Master finger:** one scan shows `CONFIRM?` ; a second scan of the same finger within 8 s reopens the captive portal.

7.3 Enrollment job lifecycle

A job is created `pending` . `/get-enrollment-job` hands it to the scanner and moves it to `in_progress` , stamping `dispatched_at` and incrementing `attempts` . The scanner enrolls the finger and POSTs the result; `/update-enrollment-job` moves it to `completed` with the FID, or `failed` with `last_error` . If the result is lost — flaky TLS, or a reboot between `storeModel()` and the POST — the job is re-delivered on a later poll, and auto-failed after a capped number of attempts. The `enrollment_jobs_status_guard` trigger stops a late or raced write from pulling a terminal job back to `in_progress` . The dashboard offers Retry and Cancel for stuck jobs, and Re-queue (optionally with overwrite) for failed ones.

7.4 Revocation and decommissioning

Set `devices.revoked = true` for an instant lockout: the scanner's next `/log-attendance` call gets a 401. For a permanent wipe, insert a `device_resets` row — deleting a device on the dashboard does this. `/get-enrollment-job` then returns `decommissioned: true` on every poll (the row isn't consumed, so it survives races), and the scanner wipes its identity and reboots into provisioning.

8 Edge functions

Six Deno functions under `backend/supabase/functions/` . All are configured `verify_jwt = false` and do their own authentication.

Function	Caller	Auth	Purpose
<code>/register</code>	Scanner, first boot	<code>x-bootstrap-secret</code>	Create an unassigned device row and issue a provisioning token
<code>/assignment-poll</code>	Scanner, while pending	<code>x-bootstrap-secret</code> + token	Once assigned, return the device secret, institution, and display name
<code>/log-attendance</code>	Scanner, every scan	<code>x-device-secret</code>	Authenticate, resolve the member in-tenant, apply the rules, dedupe, insert
<code>/get-enrollment-job</code>	Scanner, every 10 s	<code>x-device-secret</code>	Return the next job and device config; re-deliver stuck jobs; signal decommission
<code>/update-enrollment-job</code>	Scanner, after enrolling	<code>x-device-secret</code>	Record completed or failed, and the FID; institution derived from the device row
<code>/mark-absent</code>	pg_cron, 21:00 UTC daily	<code>x-cron-secret</code>	Insert absences per institution and timezone; batches of 8 institutions; needs pg_net

The cron job reads the project URL and cron secret from Vault (`supabase_project_url` , `cron_secret`) each time it fires, so no credential appears in migration history.

9 Dashboard

Next.js App Router, on a version with breaking changes from stock Next.js: middleware lives in `frontend/proxy.ts`, not `middleware.ts`. Every Supabase call is server-side through the service role.

9.1 Roles and scope

Role	Scope	Access
<code>platform_admin</code>	Every institution	All pages; creates institutions and their first super admin.
<code>super_admin</code>	One institution	Every page, including devices, enrollment, settings, and accounts.
<code>admin</code>	One institution, or one device	People, periods, promotion, attendance, the shop module; views accounts and changes only their own password. Device-bound admins are scoped to that device and may register or delete fingerprints on it.
<code>teacher / staff</code>	One device	Read-only attendance and roster for that device. An unresolved assignment shows an empty state, never everything.
<code>cashier</code>	One shop	Clients, sales, and the catalog (view only).

`dal.ts` exports `verifySession()`, `requireRole(...)`, `getInstitution()`, `resolveInstitutionScope()`, and `resolveDeviceScope()`. `verifySession()` is `cache()`-wrapped and sends suspended or deactivated tenants to `/suspended`. `node scripts/check-rbac.mjs` fails CI if any dashboard page lacks a `requireRole()` call.

9.2 Pages

Route	Description
<code>/</code>	Overview — today's present and absent counts, rate, and recent scans; takings and stock for shops; a cross-tenant summary for platform admins.
<code>/attendance</code>	Records, Summary, and By-member tabs; date, period, type, status, member, and unit filters; Late / Early badges; CSV export.
<code>/members</code> , <code>/staff</code>	Rosters with add, CSV import, edit, fingerprints, and activate / deactivate.
<code>/devices</code>	Pending assignment (platform), pending setup, configure, rename, delete.
<code>/academic</code>	Periods and holidays — "Academic", "Periods & Holidays", or "Closed Days" by type.
<code>/enrollment</code>	Job queue: register, delete, clear all, register and delete master; Retry, Cancel, Re-queue, overwrite.
<code>/promotion</code>	Year-end bulk promotion; schools only.
<code>/settings</code>	Identity, branding, labels, scan modes, tracked weekdays, punctuality, timezone, time format, name on device, and shop modules.
<code>/sales</code> , <code>/clients</code> , <code>/catalog</code> , <code>/rewards</code> , <code>/reports</code>	The retail and loyalty module; shops only.
<code>/institutions</code> , <code>/onboarding</code>	Tenant management; platform admins only.
<code>/users</code>	Accounts: email search and role filter.
<code>/unauthorized</code> , <code>/suspended</code>	Role-denied and tenant-suspended screens.

9.3 Installable PWA

A web manifest (`app/manifest.ts` , served as `/manifest.webmanifest`) and a minimal service worker (`public/sw.js`) make the dashboard installable. The worker only passes requests through — it never caches, so the authenticated dashboard can't serve stale data — and is itself served no-cache. `proxy.ts` excludes `/api` and the PWA assets so a signed-out browser can still read them. The in-app install prompt lives in `components/pwa/` .

10 OTA firmware updates

At boot, with WiFi up and no assignment pending, the scanner waits a random 0–120 s (from hardware entropy) so a fleet-wide power cut doesn't hit GitHub's unauthenticated rate limit at once. It then fetches `releases/latest` from `api.github.com/repos/sgariba21-beep/esp32-attendance-device` .

Tags must be `firmware-v<major>.<minor>.<patch>` (`OTA_TAG_PREFIX`). Only a version strictly newer than the compiled `FIRMWARE_VERSION` installs. The release's first `.bin` asset is streamed through the ESP32 `Update` library while the sensor light breathes red, and the scanner reboots into the new image. The API and CDN

hosts must be in the TLS bundle (section 5.1).

11 Key file locations

Path	Purpose
<code>firmware/ClassAttendance_Current_RTC/ClassAttendance_Current_RTC.ino</code>	The firmware sketch
<code>firmware/ClassAttendance_Current_RTC/certs.h</code>	TLS root and intermediate bundle
<code>firmware/ClassAttendance_Current_RTC/secrets.h</code>	<code>BOOTSTRAP_SECRET</code> — gitignored; copy from <code>secrets.example.h</code>
<code>hardware/PCBs/Attendance_System_v1/</code>	Carrier PCB (KiCad); <code>_v0</code> is the earlier revision
<code>hardware/CAD files/</code>	Enclosure source (FreeCAD) and print-ready 3MF parts
<code>backend/supabase/functions/</code>	The six edge functions
<code>backend/supabase/migrations/</code>	Schema migrations — applied to cloud by hand
<code>backend/supabase/config.toml</code>	Edge-function config (<code>verify_jwt = false</code> for all)
<code>frontend/proxy.ts</code>	Next.js middleware (not <code>middleware.ts</code>)
<code>frontend/lib/supabase/dal.ts</code>	Session, role, and scope helpers
<code>frontend/lib/supabase/ownership.ts</code>	<code>ownsRecord()</code> — the per-tenant ownership check
<code>frontend/app/manifest.ts</code> , <code>frontend/public/sw.js</code>	PWA manifest and service worker
<code>docs/source/</code>	Sources for this document and every manual, plus the build script
<code>docs/for-clients/</code> , <code>docs/for-operators/</code>	The built PDFs, and the operator runbooks



NOTE

This document is built from `docs/source/Technical_Documentation.md` . Update it whenever firmware contracts, the schema, or the edge-function API change, then rebuild with `python build.py` .